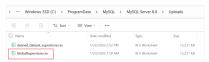
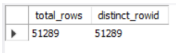
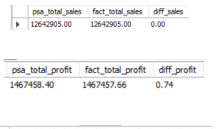
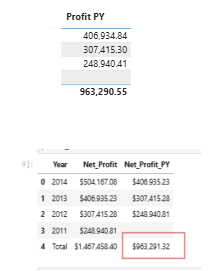
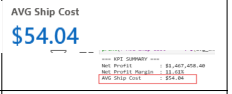
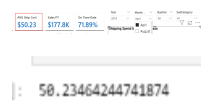
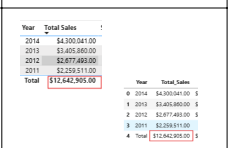
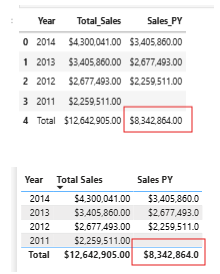
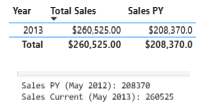
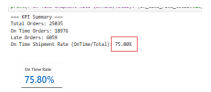
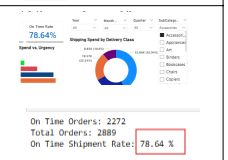
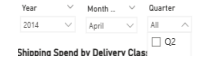
TC_No	Type	Test Scenario	Test Step	Module	Existing Result	Expected Result	Test Result	Impacted KPI (if defect found)	Fixing	Root Cause	Evidence
TC_01	Functional	**Source Data Preparation and Validation** The Global Superstore CSV file is prepared and validated as the initial data source for the data warehouse pipeline.	- Verify that the GlobalSuperstore.csv file exists in the source directory. - Open the file and confirm the presence of required columns with right format	csv	The source file is available, readable, and contains all required field.	The source file is available, readable, and contains all required field.	**Passed**	-	-	-	
TC_02	Functional	**Data Preprocessing and PSA Ingestion** The raw dataset is cleaned and preprocessed using Python, then loaded into the Persistent Staging Area (PSA) in MySQL.	- Place the cleaned file in the MySQL upload directory. - Execute the LOAD DATA INFILE command to load data into psa_superstore_dev. - Compare record counts between the cleaned CSV and PSA table.	My SQL Workbench	record counts csv and PSA table = 51289	record counts csv and PSA table = 51289	**Passed**	-	-	-	
TC_03	Functional	**Data Warehouse Transformation and Validation** Data from the PSA is transformed and loaded into the data warehouse schema, including dimension tables and the Fact_Sales table.	- Execute dimension table scripts (Date, Product, Customer, Geography). - Execute the Fact_Sales loading script. - Validate foreign key relationships between Fact_Sales and dimension tables. - Compare aggregated Sales and Profit between PSA and Fact_Sales.	My SQL Workbench	Orphan record count for all foreign key checks equals 0. Sales difference between PSA and Fact_Sales equals 0. Profit difference between PSA and Fact_Sales is 0.74	Orphan record count for all foreign key checks equals 0. Sales difference between PSA and Fact_Sales equals 0. Profit difference between PSA and Fact_Sales is 0	**Passed** (0.74 represent 0.00005% of total profit and is therefore negligible and within acceptable tolerance limits)	-	-	# Root Cause Analysis: **PSA layer:** Profit is stored as a VARCHAR. During testing, it is aggregated after casting to DECIMAL (18,6), which preserves higher precision and does not introduce row-level rounding. **Fact_Sales layer:** Profit is stored as DECIMAL (10,2). Each record is rounded to two decimal places at insert time. **Result:** Small per-row rounding effects accumulate across many records, leading to a minor total difference (0.74) between the PSA and Fact_Sales aggregates.	
TC_04	Functional	**BI Layer and KPI Validation** Data warehouse outputs are consumed by the BI layer to generate dashboards and KPIs.	Refresh the Power BI dataset to ensure the latest data is loaded. Verify that all KPI cards are displayed on the dashboard (Net Profit Margin, Net Profit, Sales PY, Average Shipping Cost, On-Time Rate).	Power BI	5 KPI Cards displayed on the Dashboard	5 KPI Cards displayed on the Dashboard	**Passed**	-	-	-	
TC_05	Functional	**BI Layer and KPI Validation** Data warehouse outputs are consumed by the BI layer to generate dashboards and KPIs.	Apply time filters (Year, Quarter, Month) and confirm that all KPI cards update accordingly.	Power BI	All KPI cards updated	All KPI cards updated	**Passed**	-	-	-	before:  after: 
TC_06	Functional	**BI Layer and KPI Validation** Data warehouse outputs are consumed by the BI layer to generate dashboards and KPIs.	Apply product filters (Category / Sub-Category) and confirm taht all KPI cards update accordingly	Power BI	All KPI cards updated	All KPI cards updated	**Passed**	-	-	-	before:  after: 
TC_07	Data	Raw Data vs BI KPI Reconciliation	- Check the value of Net Profit Margin on the BI dashboard - Compare it with the value calculated independently in Jupyter Notebook using raw data	Power BI - Operational	11.61%	11.61%	**Passed**	-	-	-	
TC_08	Data	Raw Data vs BI KPI Reconciliation	- Check the value of 'Total Net Profit' on the BI dashboard - Compare it with the value calculated independently in Jupyter Notebook using raw data	Power BI - Operational	1.47 M >> 1,467,457.66	1,467,458.40	**Passed** (there's discrepancy between exisiting and expected result, but already	-	-	# Root Cause Analysis: The Net Profit difference is caused by numeric precision and row-level rounding of the Profit field. Python sums raw floating-point values, while Power BI applies fixed decimal or currency rounding.	

TC_09	Functional	Dashboard Slicer Functionality Validation	- Verify the 'Total Net Profit' KPI on the dashboard - Apply the following slicer settings: Year = All Month = All Quarter = Q1 Sub-Category = All - Observe whether the KPI value updates accordingly	Power BI - Operational	$238.6K	$238,555.10	**Passed**	-	-	-	
TC_10	Data	Raw Data vs BI KPI Reconciliation	- Check the value of 'Total Net Profit PY' on the BI dashboard - Compare it with the value calculated independently in Jupyter Notebook using raw data	Power BI - Logistics & Profit	Total Net Profit PY = 991,291.93 **After fixing:** 963,290,55	Total Net Profit PY' = 963291.32	**Failed** **Update: Passed.** (there's discrepancy between exisiting and expected result, but already explained in the root cause analysis #1)	1. Card 'Total Net Profit PY' - Dashboard 'Operational' 2. Table 'Year - Net Profit - Profit PY' - Dashboard 'Logistic & Profit'	Update Measures (**Profit PY**): IF(ISBLANK([Net Profit]), BLANK(), CALCULATE([Net Profit], SAMEPERIODLASTYEAR ('db_global_superstore_dev dim_date'[FullDate]))) to **Profit PY (Fixed) =** SUMX(FILTER(VALUES('db_global_superstore_dev dim_date'[Year]), NOT ISBLANK(CALCULATE([Net Profit]))), CALCULATE([Net Profit], SAMEPERIODLASTYEAR ('db_global_superstore_dev dim_date'[FullDate])))	# Root Cause Analysis: 1. The Net Profit difference is caused by numeric precision and row-level rounding of the Profit field. Python sums raw floating-point values, while Power BI applies fixed decimal or currency rounding. 2. The old measure applies SAMEPERIODLASTYEAR directly to the visual context. As a result, on the Total row Power BI shifts the entire date range at once (2011–2014 → 2010–2013), causing the Total PY value to expand and include unintended years.	
TC_11	Data	Raw Data vs BI KPI Reconciliation	- Check the value of 'AVG Ship Cost' on the BI dashboard - Compare it with the value calculated independently in Jupyter Notebook using raw data	Power BI - Operational	54.04	54.04	**Passed**	-	-	-	
TC_12	Functional	Dashboard Slicer Functionality Validation	- Verify the 'AVG Ship Cost' KPI on the dashboard - Apply the following slicer settings: Year = 2014 Month = April Quarter = All Sub-Category = All - Observe whether the KPI value updates accordingly	Power BI - Operational	50.23	50.23	**Passed**	-	-	-	
TC_13	Data	Raw Data vs BI KPI Reconciliation	Check value 'Total Sales' on the BI Dashboard. Compare it with the value calculated independently in Jupyter Notebook using raw data.	Power BI - Logistics & Profit	- Year 2011 = 2,259,511 - Year 2012 = 2,677,493 - Year 2013 = 3,405,860 - Year 2014 = 4,300,041 Total: 12,642,905.00	12,642,905.00	**Passed**	-	-	-	
TC_14	Data	Raw Data vs BI KPI Reconciliation	Check value 'Total Sales PY' on the Bi Dashboard. Compare it with the value calculated independently in Jupyter Notebook using raw data.	Power BI - Operational	Total 'Sales PY' = 8,584,131 **After fixing**: 8,342,864.00	Total 'Sales PY' = 8,342,864.00	**Failed** **Update: Passed**	1. Card 'Sales PY' - Dashboard 'Operational' 2. Table 'Year - Total Sales - Sales PY' - Dashboard 'Logistic & Profit'	Update Measures (**Sales PY**): IF(ISBLANK([Total Sales]), BLANK(), CALCULATE([Total Sales], SAMEPERIODLASTYEAR ('db_global_superstore_dev dim_date'[FullDate]))) **Sales PY (Fixed) =** SUMX(FILTER(VALUES('db_global_superstore_dev dim_date'[Year]), NOT ISBLANK(CALCULATE([Total Sales]))), CALCULATE([Total Sales], SAMEPERIODLASTYEAR ('db_global_superstore_dev dim_date'[FullDate])))	The old measure applies SAMEPERIODLASTYEAR directly to the visual context. As a result, on the Total row Power BI shifts the entire date range at once (2011–2014 → 2010–2013), causing the Total PY value to expand and include unintended years.	

TC_15	Functional	Dashboard Slicer Functionality Validation	- Verify the 'Total Sales PY' KPI on the dashboard - Apply the following slicer settings: Year = 2013 Month = May Quarter = Q2 Sub-Category = All - Observe whether the KPI value updates accordingly	Power BI - Operational	208.4K	208,370	**Passed**	-	-	-	
TC_16	Data	Raw Data vs BI KPI Reconciliation	Check value 'On Time Shipping Rate' on the Bi Dashboard. Compare it with the value calculated independently in Jupyter Notebook using raw data.	Power BI - Operational	75.80%	75.80%	**Passed**	-	-	-	
TC_17	Functional	Dashboard Slicer Functionality Validation	- Verify the 'On Time Rate' KPI on the dashboard - Apply the following slicer settings: Year = All Month = All Quarter = All Sub-Category = Accessories - Observe whether the KPI value updates accordingly	Power BI - Operational	78.64%	78.64 %	**Passed**	-	-	-	
TC_18	Functional	Dashboard Slicer Functionality Validation	Select slicer Year: 2014,Month: April.	Power BI	Slicer on Quarter only Show 'Q2"	Slicer on Quarter only Show 'Q2"	**Passed**	-	-	-	
TC_19	Functional	Dashboard Slicer Functionality Validation	Slicer Year shows 2011 - 2015	Power BI	2011-2015	2011-2015	**Passed**	-	-	-	
TC_20	Functional	Dashboard Slicer Functionality Validation	Slicer Month Name shows 12 Month Names	Power BI	Jan - Dec	Jan - Dec	**Passed**	-	-	-	